

NetSage AI

An AI-assisted network fault triage tool with a mandatory human review step

Submitted by	Danish Soreng
Institution	KIIT University
Stack	Python 3.12 · Flask · SQLAlchemy · Apache Airflow · Anthropic Claude API
Deliverable	Working web application, batch pipeline and 30-case evaluation set

1. The problem

When a user reports that "the network is down", they have described a symptom, not a fault. Behind that single sentence sit at least eight materially different causes — a port left in the wrong VLAN, a DHCP scope that never answered, an access list denying the traffic, a router with no route to the destination, a misconfigured default gateway, a NAT translation that is missing, a failed DNS lookup, or a physically dead port. From outside the network they are indistinguishable.

An experienced engineer narrows this to one or two candidates in seconds, because they know which diagnostic command separates them. A junior engineer knows the same commands but not the ordering, so they work through them in the order they happen to recall. On a first-line service desk this is where the bulk of the handling time on a connectivity ticket is spent, and it is a training problem rather than a tooling problem.

The industry need: reduce time-to-first-hypothesis for inexperienced network staff, without introducing a system that can confidently send them in the wrong direction.

2. Proposed solution

NetSage AI accepts a fault description in plain language together with whatever show command output has already been collected. It returns a single most-likely root cause, the specific lines of evidence that support it, the one command that would confirm or eliminate it, and the remediation steps — then requires a person to accept, edit or reject that proposal before it counts as an answer.

2.1 The governing design decision

The AI never has the final word. Every proposal is stored as *pending* until a human records a decision. There is no code path in the system by which a model output becomes the accepted answer on its own, and no path that applies a configuration change to a device.

This is an engineering safeguard as much as an ethical one. A language model produces fluent, confident text whether or not it is correct. A tool that acted on that output unsupervised would be worse than no tool, because its intended user is precisely the person least equipped to catch its mistakes. Making the human decision mandatory is what makes the system safe to deploy to that user.

2.2 Two independent checkers, run in a fixed order

A deterministic rule checker runs *before* the model, and its findings are passed into the model prompt as established facts. The ordering is deliberate: run afterwards, the findings would be merely a contradiction for the reviewer to notice; run beforehand, they constrain what the model is able to conclude.

	Deterministic rule checker	AI assistant
Method	Fixed rules in plain Python	Claude, reasoning over the case text
Coverage	Eight specific mechanical faults	Broad, in principle unbounded
Failure mode	Only if the rule itself is wrong	Can be wrong, and wrong confidently
Determinism	Identical output for identical input	Not guaranteed

Neither is sufficient alone: the rules are reliable but narrow, the model is broad but fallible. Running both means an incorrect model answer is contradicted on the same screen by a component that cannot fabricate.

3. System architecture

Four stages, executed in order. Two entry points share one implementation: the web console submits a single case interactively, while an Airflow DAG processes the case directory on an hourly schedule. Both call the same functions and write to the same tables — the diagnosis logic is not duplicated anywhere in the codebase.

Stage	What happens	Writes to
01 Ingest	Symptom, topology notes and raw command output are captured	cases
02 Rule check	Eight deterministic checks run. No model involved	rule_findings
03 AI diagnosis	Claude is called with the case <i>and</i> the findings as facts	diagnoses
04 Human review	A person accepts, edits or rejects. The only closing action	reviews

3.1 Database design

Four tables, one per stage. The separation of diagnoses from reviews carries a specific argument: if a human correction overwrote the model proposal, the record of the disagreement would be destroyed, and the question "how often was the assistant correct?" could never be answered afterwards. Keeping them apart means both answers persist side by side.

Every diagnosis row additionally records an engine field — `claude` or `heuristic` — so output from the two sources is never conflated when results are aggregated.

4. The deterministic rule checker

Implemented in `app/rule_checker.py` as plain pattern matching and address arithmetic. No model, no network calls, no randomness. Identical input always yields identical findings.

Check	Sev.	Detects
<code>DUPLICATE_IP</code>	high	The same IPv4 address configured on two devices
<code>INVALID_SUBNET_MASK</code>	high	A mask that is not a contiguous run of 1-bits, found by binary inspection
<code>GATEWAY_OFF_SUBNET</code>	high	A default gateway outside the host's own subnet, computed from address and mask
<code>APIPA_ADDRESS</code>	high	A 169.254.x.x address, which is proof that no DHCP server replied
<code>INTERFACE_DOWN</code>	high	An interface reported administratively down or down/down
<code>NO_DEFAULT_ROUTE</code>	med.	A routing table reporting no gateway of last resort

Check	Sev.	Detects
VLAN_SUBNET_MISMATCH	med.	Devices sharing a VLAN but sitting on different subnets
DNS_RESOLUTION_FAILURE	med.	A failed name lookup alongside a successful IP ping — both required

Each rule is isolated, so a failure in one cannot prevent the other seven from running; the failure is recorded as a finding instead.

5. The AI diagnosis stage

Implemented in `app/ai_assistant.py` using the official Anthropic Python SDK against `claude-opus-5`. The request carries the symptom, the topology notes, the raw command output, and the deterministic findings presented explicitly as facts to be reasoned with rather than around.

5.1 Schema-enforced output

The response is constrained by a structured output schema rather than requested in prose. The API itself enforces the seven fields, the 0–1 confidence range, and the permitted vocabulary for fault type and OSI layer:

```
class ProposedDiagnosis(BaseModel):
    root_cause: str
    fault_type: Literal["VLAN", "DHCP", "DNS", "ROUTING",
                       "ACL", "NAT", "PHYSICAL", "OTHER"]
    osi_layer: Literal["Layer 1" ... "Layer 7"]
    confidence: float = Field(ge=0.0, le=1.0)
    evidence: str
    next_command: str
    fix_steps: List[str]
```

A proposal therefore cannot arrive malformed, partially populated, or in a fault category the reporting layer does not recognise. This is what makes the category-agreement metric meaningful rather than an artefact of string cleanup.

Two of these fields exist to make the proposal **falsifiable**: `evidence` exposes the reasoning for inspection, and `next_command` tells the engineer how to prove the proposal wrong.

5.2 Offline fallback

Where no API key is configured, or any API call fails, a local heuristic engine built from keyword signatures and the rule findings takes over, so the pipeline never stalls and the system demonstrates end to end with no external dependency. The substitution is not concealed: the engine is recorded on the row and displayed on screen.

6. The application

Screen	Purpose
/	Triage console. Intake form, live rule findings, the AI proposal with confidence and evidence, the accept / edit / reject control, a case-grounded assistant for follow-up questions, and a session review log
/overview	Pipeline overview. Counters for all four tables, the pending review queue, a spotlight case, and the correction log — every figure read live from the database on load
/queue	Review queue. All cases with review state, colour-coded by OSI layer
/case/<ref>	Case detail. Evidence, findings, proposal and the full review form; for the evaluation set, the recorded ground truth

Screen	Purpose
/dashboard	Metrics. Faults by type and severity, decision breakdown, agreement rate, and the responsible-AI correction log

A JSON API (app/api.py) sits behind the console, exposing triage, review, chat, metrics and health endpoints.

7. Responsible AI in practice

The responsible-AI properties of this system are structural rather than advisory — they are consequences of how the data model and the control flow are arranged.

- **Mandatory human gate.** No proposal is ever marked correct without a recorded human decision.
- **Automatic correction log.** Every edit or rejection appears in the log as a by-product of doing the review. Nobody has to remember to file anything, which is the only way such a record survives in a busy team.
- **Provenance on every row.** The engine that produced each diagnosis is stored, so heuristic and model output are never aggregated together unknowingly.
- **Falsifiability by construction.** Every proposal must cite its evidence and name a command that would disprove it.
- **Calibrated uncertainty.** Confidence below 0.55 is flagged in the interface. Where no deterministic check fires, the interface states explicitly that the proposal rests on the symptom text alone.
- **Declared scope.** The console footer permanently displays that the system reads pasted output rather than live devices, and proposes fixes rather than applying them.

8. Results and verification

Verified	Outcome
Ingest of the evaluation set	30 cases loaded from JSON
Deterministic checking	17 findings raised across the 30 cases
AI diagnosis stage	30 proposals produced, one per case
End-to-end review loop	Case submitted through the browser, checked, diagnosed, persisted to all four tables; decision recorded and reflected in the metrics
API surface	13 routes verified by an automated smoke test from an empty database
Checker parity	The browser fallback checker and the server checker produce identical findings on identical input

On the agreement figure. The dashboard reports a category-match rate against the recorded ground truth. This is measured on the same 30 cases the system was developed against, and therefore demonstrates consistency on known cases. It is **not** a measure of real-world accuracy, and the interface states this next to the number rather than in a footnote.

9. Limitations

- The system reads command output that has been collected and pasted in. It does not connect to live network equipment.
- It proposes remediation; it does not apply configuration changes.
- The rule checker's text parsing assumes the output layout used in the evaluation set. Real device output varies by vendor and software version and would require the patterns to be widened.

- Accuracy on genuinely unseen faults is unmeasured. Measuring it would require running the system against real tickets over a period.
- The 30-case evaluation set is constructed for a laboratory, not sampled from a production service desk.
- The PostgreSQL and Airflow scheduler paths are implemented and the DAG compiles, but have not been executed in the development environment; the demonstration runs on SQLite.

10. Further work

- Evaluate against real service-desk tickets to obtain an accuracy figure with external validity.
- Widen the rule checker to parse output formats from additional vendors and software versions.
- Use the accumulated correction log to identify fault categories the assistant handles poorly, then either add deterministic rules for them or constrain what the assistant is permitted to claim in those categories.
- Execute the PostgreSQL and Airflow deployment paths to complete the production configuration.

11. Running the system

```
pip install -r requirements.txt
copy .env.example .env      # add ANTHROPIC_API_KEY (optional)
python run.py               # http://127.0.0.1:5000
```

The entry point creates the schema, seeds the evaluation set from data/cases/*.json on first run only, and serves the application. Without an API key the system runs complete on the local heuristic engine, so it can be demonstrated with no external dependency and no cost.

NetSage AI · four tables: cases, rule_findings, diagnoses, reviews · the AI proposes, a person decides